

FlashInfer deep dive: sparse KV, JIT, load balancing

NVIDIA A10 (sm_86, 72 SMs, 600 GB/s peak) · FlashInfer 0.6.18 · torch 2.13.0+cu130 (CUDA 13.0) · Qwen3-4B attention shapes: 32 query heads, 8 KV heads, head_dim 128

2a · How block-sparse KV memory works

The concept behind the 2a measurements: the KV cache is a pool of pages, and each request is a sparse row of page indices into it.

Physical page in KV pool

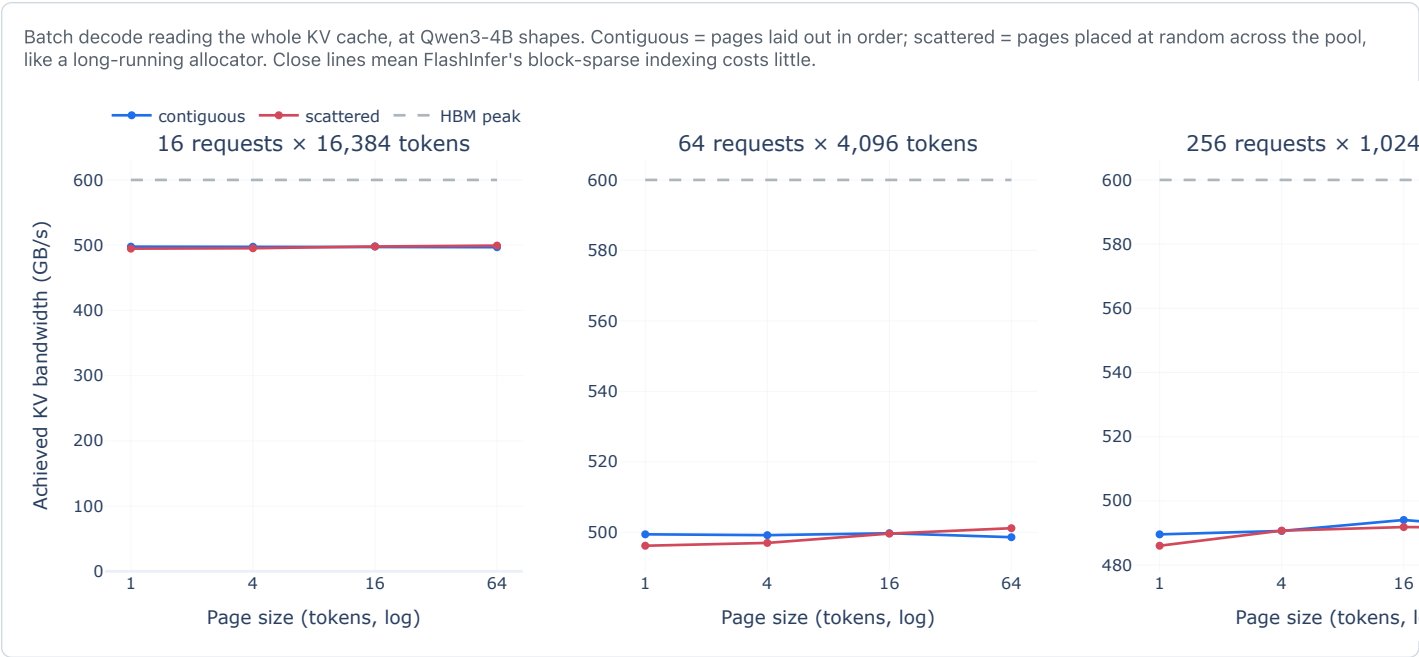
	0	1	2	3	4	5	6	7	8	9
Req A (agent 1)										
Req B (agent 2)										
Req C (chat)										

`kv_indptr` = [0, 3, 6, 8]
`kv_page_indices` = [7, 2, 5 | 7, 2, 9 | 4, 1]
`kv_last_page_len` = [5, 16, 9] (page_size 16)

Shared prefix page (prompt + tools)
 Private page (one request)
 Not used by this request

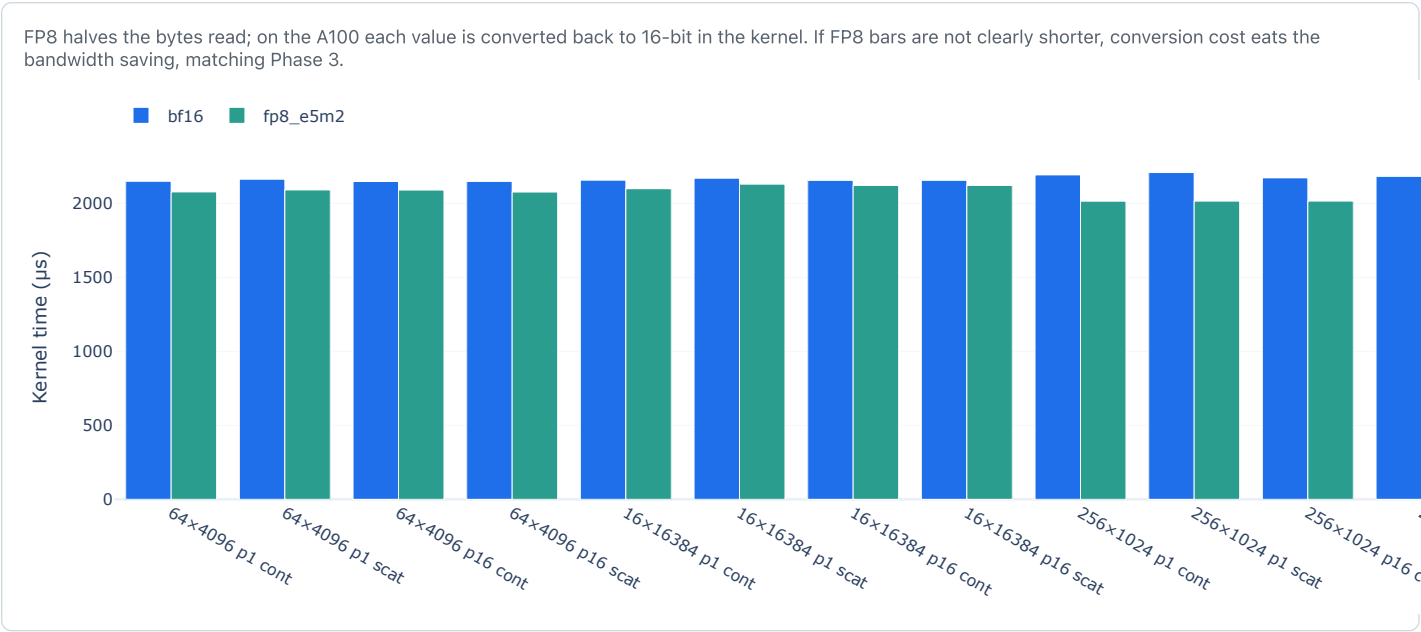
How it works: SGLang allocates KV pages anywhere in one pool and builds these three index arrays each step; FlashInfer reads only them. Requests A and B point at the same pages 7 and 2, so a shared prefix is stored once. Each page entry is a contiguous ~2 KB read per layer, which is why scattered pages cost little (tested below).

2a · Sparse paged KV: does scattering pages cost bandwidth?



layout	batch	kv_len	page_size	contiguous	scattered	scattered / contiguous
	256	1024	64	491.0	492.0	100%

2a · FP8 vs BF16 KV: kernel time

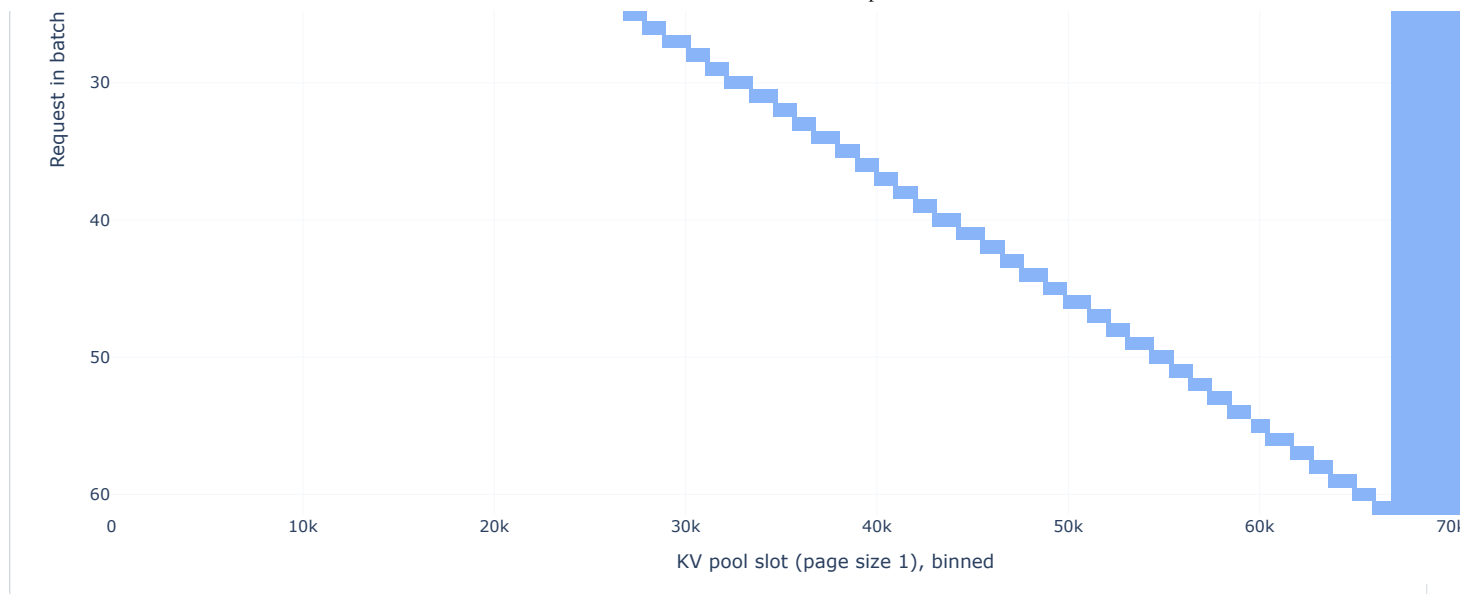


2a · Bookkeeping: index entries per batch



2a · Live KV index map (decode batch of 62 requests)





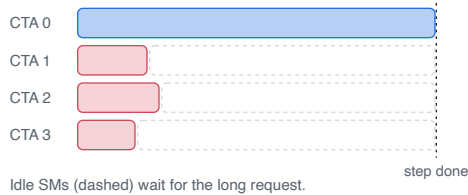
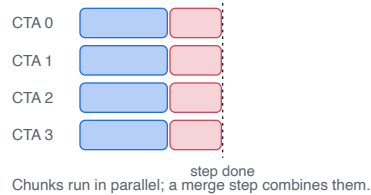
2a · KV layout over time

35 sampled plan() calls. Falling contiguous runs mean the allocator is fragmenting the pool; FlashInfer's indexing makes that harmless if 2a's bandwidth lines match.



2c · How FlashInfer balances work across the GPU

The concept behind the 2c measurements: a scheduler splits long requests so every SM finishes together.

Naive: one request per CTA**FlashInfer: split long request, pack lanes****plan() on CPU, once per step**

input: this step's KV lengths
output: CTA work queues

GPU workspace buffer

plan copied asynchronously
CUDA-graph safe

run() + merge

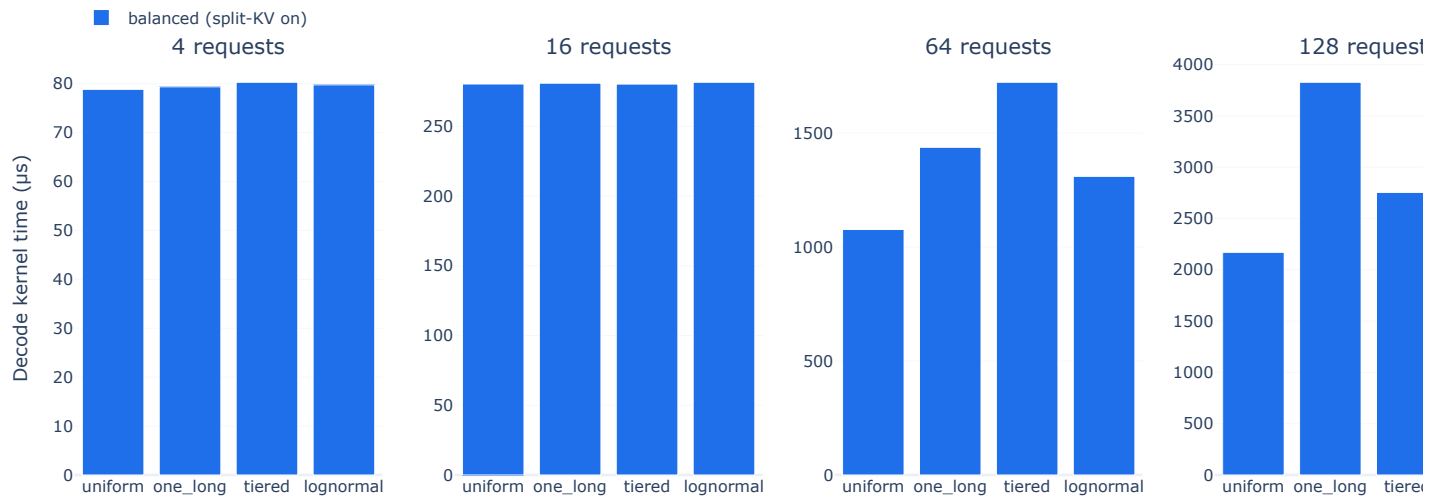
persistent kernel reads plan
reused by all 36 layers

How it works: lengths change every decode step, so FlashInfer re-plans each step on the CPU, then every layer reuses that plan. Tested below: kernel time with split-KV on vs off for skewed batches, and plan() cost per layer.

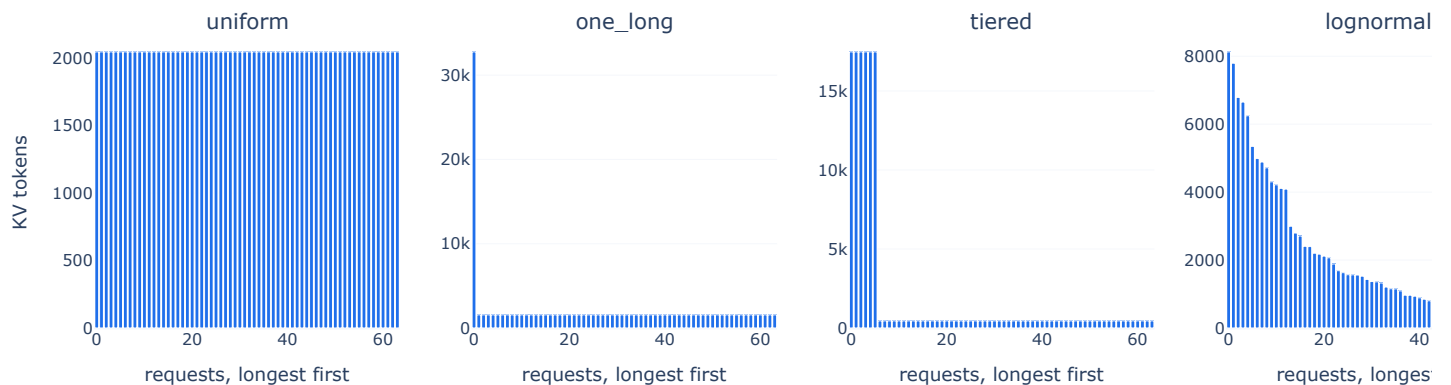
Blue = the long request; red = short requests. Illustrative widths.

2c · Load balancing: skewed batches with split-KV on vs off

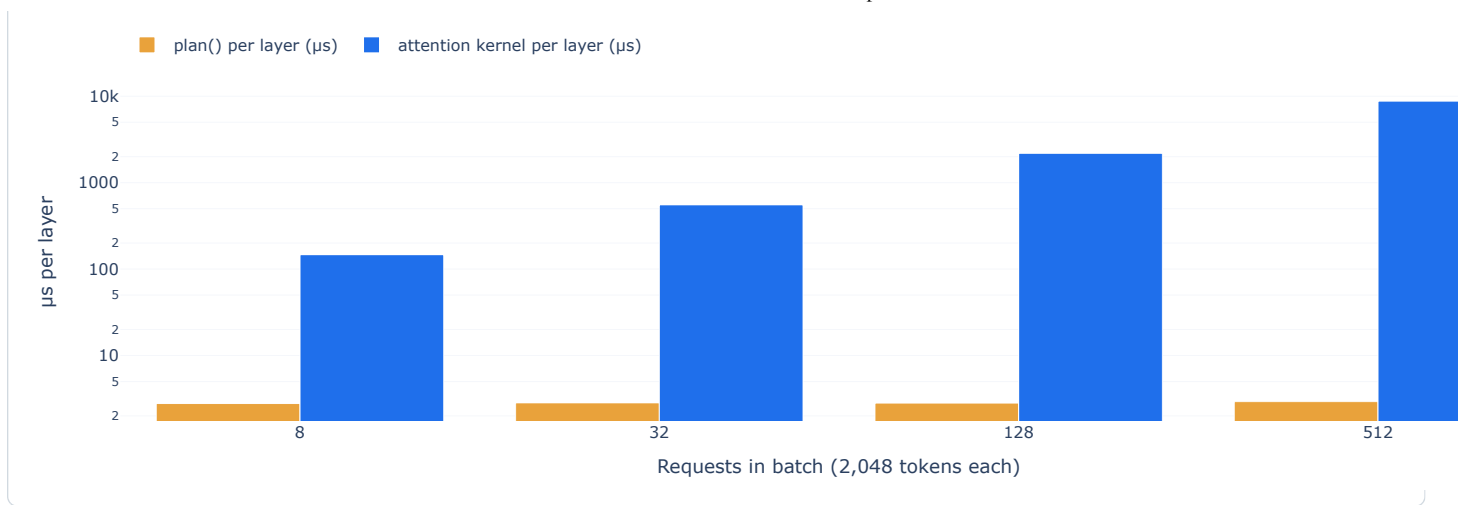
Same total tokens per batch (2,048 per request on average), different length mixes. Without split-KV, a long request's work cannot be spread over idle SMs, so the batch waits for it.

**2c · What the 64-request batches look like**

Request lengths in each distribution.

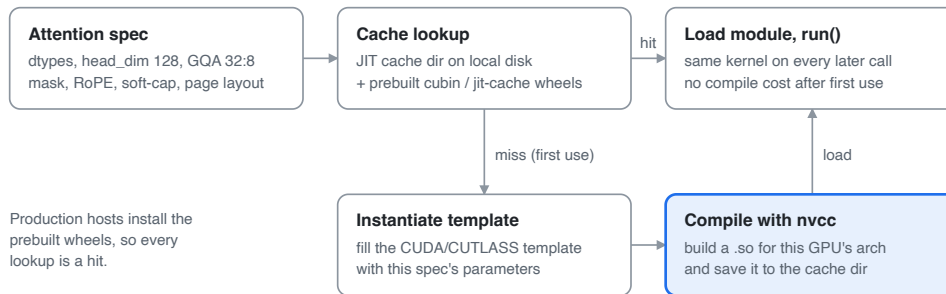
**2c · Scheduling overhead: plan() vs the kernel**

plan() runs once per step on the CPU and is reused by all 36 layers, so its cost is shown per layer.



2b · How JIT compilation of attention kernels works

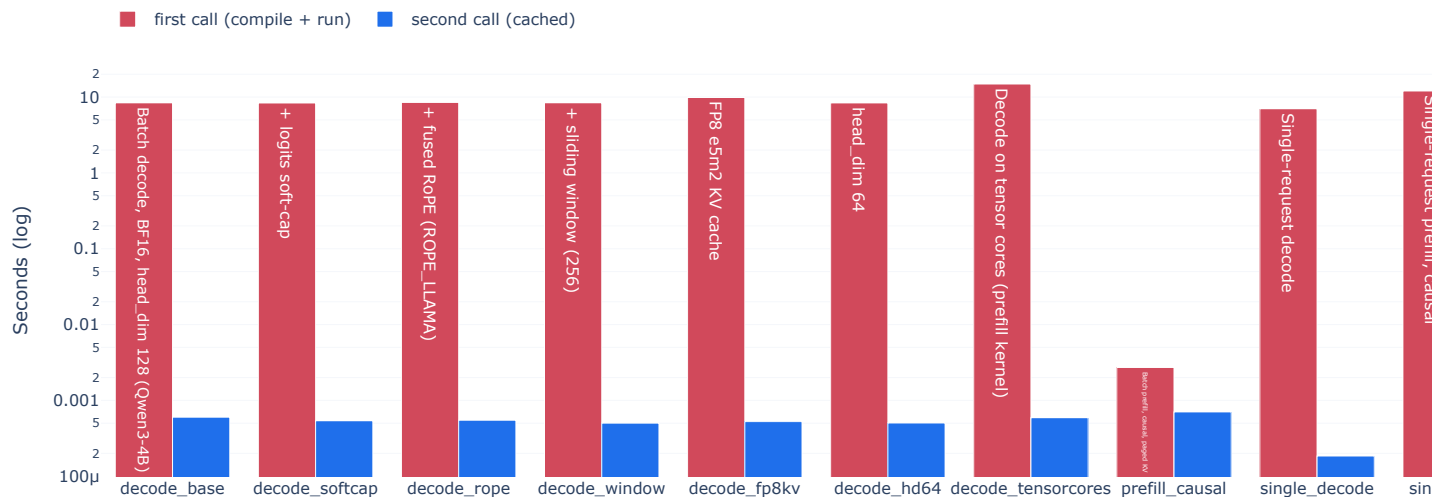
The concept behind the 2b measurements: each attention variant is compiled from a template on first use.



How it works: FlashInfer keeps one template per kernel family. Each new combination of options is a new variant, compiled once (the blue step, timed below as 'first call') and reused from disk afterwards ('second call').

2b · JIT: what each new attention variant costs the first time

Each bar is one kernel specialization. The first call instantiates the template, compiles it with nvcc and caches the module; later calls reuse it. Prebuilt wheels skip the red bars entirely.



Modules built per variant. Watched: /home/ubuntu/fi-cold/jit-workspace, /home/ubuntu/fi-cold/lib/python3.12/site-packages/flashinfer/data/aot, /home/ubuntu/fi-cold/lib/python3.12/site-packages/flashinfer/data/csrc, /home/ubuntu/fi-cold/lib/python3.12/site-packages/flashinfer/data/include, /home/ubuntu/fi-cold/lib/python3.12/site-packages/flashinfer/data/spdlog/include

variant	description	first_s	cached_ms	new_files	
decode_base	Batch decode, BF16, head_dim 128 (Qwen3-4B)	8.4	0.6	12	batch_decode_with_kv_cache_dtype_q_bf16_dtype_kv_bf16_dtype_o_bf16_dtype_idx_i32_head
decode_softcap	+ logits soft-cap	8.4	0.5	12	batch_decode_with_kv_cache_dtype_q_bf16_dtype_kv_bf16_dtype_o_bf16_dtype_idx_i32_head
decode_rope	+ fused RoPE (ROPE_LLAMA)	8.5	0.5	12	batch_decode_with_kv_cache_dtype_q_bf16_dtype_kv_bf16_dtype_o_bf16_dtype_idx_i32_head
decode_window	+ sliding window (256)	8.4	0.5	12	batch_decode_with_kv_cache_dtype_q_bf16_dtype_kv_bf16_dtype_o_bf16_dtype_idx_i32_head
decode_fp8kv	FP8 e5m2 KV cache	9.9	0.5	12	batch_decode_with_kv_cache_dtype_q_bf16_dtype_kv_e5m2_dtype_o_bf16_dtype_idx_i32_head
decode_hd64	head_dim 64	8.4	0.5	12	batch_decode_with_kv_cache_dtype_q_bf16_dtype_kv_bf16_dtype_o_bf16_dtype_idx_i32_head
decode_tensorcores	Decode on tensor cores (prefill kernel)	14.9	0.6	26	batch_prefill_with_kv_cache_dtype_q_bf16_dtype_kv_bf16_dtype_o_bf16_dtype_idx_i32_head_dim_qk_128
prefill_causal	Batch prefill, causal, paged KV	0.0	0.7	0	
single_decode	Single-request decode	7.0	0.2	12	single_decode_with_kv_cache_dtype_q_bf16_dtype_kv_bf16_dtype_o_bf16_head
single_prefill	Single-request prefill, causal	12.0	0.2	18	single_prefill_with_kv_cache_dtype_q_bf16_dtype_kv_bf16_dtype_o_bf16_head_dim_qk_128

2b · What changes between two variants

Generated source for the base decode kernel vs the soft-cap variant: only template parameters differ; the kernel body is the same template.

```
--- batch_decode_config.inc
+++ batch_decode_config.inc
@@ -14,5 +14,5 @@

#define DISPATCH_context(DType0, DTypeKV, DTypeO, IdType, HEAD_DIM_QK, HEAD_DIM_VO, POS_ENCODING_MODE, USE_SLIDING_WINDOW, USE_LOGITS_SOFT_CAP, Attention)
- using AttentionVariant = DefaultAttention<false, false, false, false>; \
+ using AttentionVariant = DefaultAttention<false, false, true, false>; \
__VA_ARGS__(); \
}
@@ -26,5 +26,5 @@
constexpr int HEAD_DIM_QK = 128;
constexpr int HEAD_DIM_VO = 128;
-constexpr auto USE_LOGITS_SOFT_CAP = false;
+constexpr auto USE_LOGITS_SOFT_CAP = true;
constexpr auto POS_ENCODING_MODE = PosEncodingMode::kNone;
constexpr auto USE_SLIDING_WINDOW = false;
```